Team 4 – Trade Fair AI Matchmaker: Final Flow

Oct 29, 2026 · @Someone

Goal and core idea

We build a two-way, explainable matchmaker. A guest fills a short form and gets the top 30 exhibitors in under 2 seconds, each with a match % and a "Matched on" reason. Exhibitors see their best-fit guests as anonymised leads, and both sides update live when a guest edits their profile.

Our edge over other teams is **mutual fit**. The score combines two checks:

- **Guest → Exhibitor:** does this stall sell what the guest wants to buy?
- **Exhibitor → Guest:** does this guest match the exhibitor's target buyer (industry, seniority, company size)?

Final score = geometric mean of the two, normalised to 0–100%. A stall ranks high only when the fit works both ways.

What we avoid: a plain "send profile to an LLM and ask it to rank" wrapper. The rubric scores that as Level 2 ("superficial wrappers").

Data setup (done before the demo)

All data is synthetic and generated by one script. It must be ready by Friday morning.

Dataset	Size	What it holds	Why we need it
Exhibitors	~200	Name, products, industry verticals, target buyer criteria, hall/booth, messy free-text description	The list every guest is matched against
Seeded guests	~500	Industry, job role, seniority, company size, region, procurement interests, free text, opt-in flag	Feeds the exhibitor portal leads and the fairness view; new form users join this pool

Dataset	Size	What it holds	Why we need it
Taxonomy	~40–60 tags	Shared industry and product categories (e.g. Industrial Automation, Component Sourcing)	Makes "Matched on" reasons consistent on both sides

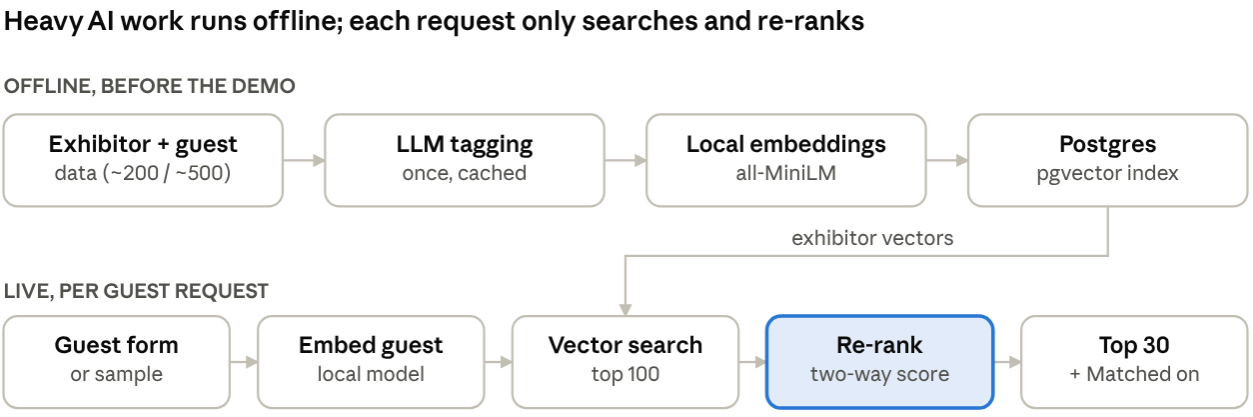
Steps:

1. Generate exhibitors and guests with an LLM script, then save as JSON and load into Postgres.
2. Run an LLM pass once to extract taxonomy tags from each free-text description. Cache the results.
3. Embed every exhibitor and guest profile with a local model (e.g. all-MiniLM) and store vectors in pgvector.

Nothing here calls the internet during the demo.

Matching engine pipeline

All slow AI work happens once, offline. A live request only embeds one guest, searches and re-ranks, which keeps it well under 2 seconds.



matching engine · offline prep and live request

The re-rank step scores each of the 100 candidates on four factors:

- **Semantic similarity:** guest vs exhibitor embeddings.
- **Interest overlap:** shared taxonomy tags. These become the "Matched on" chips.
- **Intent match:** procurement interests vs the exhibitor's products.
- **Buyer fit:** the guest vs the exhibitor's target buyer criteria.

The first three form the guest → exhibitor score; buyer fit forms the exhibitor → guest score. The final match % is their geometric mean. Each factor's value is returned so the UI can show the breakdown bars.

Guest flow (web and mobile)

The web app and the mobile app follow the same flow and call the same API.

1. **Landing page:** a short hook ("200 stalls. Find your 30 in 2 seconds.") and a "Find my stalls" button.
2. **Details form:** industry, job role, company size, procurement interests, and a free-text "what I'm looking for". Add 2–3 **"Try a sample profile"** buttons that fill the form instantly for the demo.
3. **Loading state:** a clear indicator while the engine runs.
4. **Top 30 results:** each card shows exhibitor name, match %, hall/booth, product categories and "Matched on" chips. A latency badge shows the time taken (e.g. "Top 30 in 180 ms").
5. **Search and filter:** by score range, industry, hall and product category; sort by score, hall or name.
6. **Stall detail:** opens on tap. Shows the score breakdown bars (semantic, interests, buyer fit, intent) and the full rationale.
7. **Bookmarks:** save stalls to "My visit list".
8. **Plan my route:** orders bookmarked stalls by hall and booth to cut walking.
9. **Download PDF:** exports the visit list with booth details and reasons.
10. **Edit my details:** the list recomputes and cards animate to their new ranks.

Bookmarks and profile edits sync between web and mobile because both use the same backend.

Exhibitor portal (stretch goal)

The portal reuses the same scores, read from the exhibitor's side. Build it only after the guest flow works end to end.

1. **Exhibitor picker / login.**
2. **Dashboard:** estimated footfall, lead count, and charts for seniority, purchase intent, region and company size.
3. **Lead list:** a selector for top 20, 50 or 100 guests. Rows show match %, role level and industry. Names and contacts appear only for guests who opted in.

4. **Live updates:** when a guest edits their profile on mobile, the leads and charts refresh through Socket.io.

Trust layer: fairness, privacy, fallbacks

This section earns the Responsible AI points and protects the live demo.

- **No boosts:** no sponsor, booth-size or paid ranking. Scores come only from profile fit.
- **Fairness view:** a small admin page showing how many exhibitors appear in at least one guest's top 30, and how evenly exposure is spread.
- **Privacy:** the exhibitor portal shows aggregates only. Any group with fewer than 5 guests is hidden. Contact details appear only for opted-in guests.
- **Offline fallback:** the local embedding model plus cached LLM tags mean the demo runs without internet.
- **Visible latency:** a loading indicator and the latency badge, as the brief asks.

Tech stack

Layer	Choice
Backend API	Node.js + TypeScript (Express or Fastify)
Database	PostgreSQL + pgvector
Embeddings	Local all-MiniLM (transformers.js or a small Python service)
Tag extraction	LLM, run once offline, results cached
Guest web app	React + Vite + Tailwind + shadcn/ui
Exhibitor portal	React + Recharts
Mobile PoC	Expo React Native
Real-time sync	Socket.io
PDF export	react-pdf or jsPDF

Team split and timeline

Nine people, five workstreams. Fill in owners at the kickoff call.

Workstream	People	Owns	Done by
Data and taxonomy	2	Generator script, tags, embeddings, DB seed	Fri 10 AM
Matching engine and API	2	Retrieval, re-ranking, explanations, bookmarks, Socket.io	Fri 2 PM
Guest web app	2	Landing, form, results, filters, detail, route, PDF	Fri 8 PM
Exhibitor portal and mobile	2	Portal dashboard and leads, Expo app with the same flow	Sat 2 AM
Pitch and demo	1	Slides, demo script, sample profiles, Q&A prep, fallback checks	Sat 9 AM

Key checkpoints:

- **Thu night:** schema, taxonomy and API contract agreed, so everyone builds in parallel against mock data.
- **Fri 2 PM:** engine and guest web app connected end to end.
- **Fri night:** stretch features (portal, mobile, live sync).
- **Sat 9–10 AM:** no new features. Rehearse, test the offline fallback, set browser zoom to 125%.

If time runs short, cut in this order: mobile app, then exhibitor portal. Never cut explanations, filters or the live recompute.

Live demo script and rubric mapping

The live demo slot is 3:30–6:30 of our 10 minutes. Run it in this order:

1. Open the landing page and click a sample profile. The top 30 appear with the latency badge.
2. Filter by hall or industry. Open the #1 stall and walk through its score breakdown and "Matched on" reasons.
3. Bookmark three stalls, tap "Plan my route" and download the PDF.
4. On mobile, edit the guest's interests. The guest list and the exhibitor portal update live.
5. Close on the fairness view.

Rubric category	Points	What covers it
Challenge alignment	15	All five core requirements plus the stretch goal
AI architecture	15	Enrich → embed → retrieve → re-rank → explain pipeline
Software engineering	15	One clean API used by web, mobile and portal
Live demo	15	Sample profiles, offline fallback, rehearsed script
Innovation	10	Two-way mutual-fit scoring, route planning
UX and AI interaction	10	Explanations, latency badge, animated re-ranking
Responsible AI	10	No boosts, fairness view, k-anonymity, opt-in contacts
Presentation and Q&A	10	Clear trade-offs: local embeddings vs LLM ranking, speed vs depth